

Contents

1. Objective	2
2. Introduction	2
3. Literature Review	2
4. Components Required	3
5. Circuit Description	4
5.1 Power Distribution	4
5.2 Motor Driver Interface	4
5.3 Input System	4
5.4 Output Indicators.....	4
6. Circuit Operation.....	5
7. Arduino Code	6
8. Circuit Working	7
1. System Initialization.....	7
2. Idle State (No Input Condition)	7
3. Forward Operation.....	8
4. Speed Control using PWM	8
5. Reverse Operation	8
6. Motor Stop Condition	8
7. System Stability and Protection	9
8. Overall Working Summary	9
9. Relevant Equations	9
10. Observations / Results	9
10. Analysis	10
11. Experiential Learning	10
12. Conclusion	10

Experiment Title: H-Bridge DC Motor Control using Arduino

1. Objective

- To design and implement a motor control system using an H-Bridge.
- To control motor speed using PWM signals.
- To control motor direction using digital logic.
- To analyze motor behavior at different duty cycles

2. Introduction

DC motors are widely used in automation, robotics, and control systems due to their simplicity and efficiency. Controlling both **speed and direction** of a DC motor is an essential requirement in many applications.

In this project, an H-Bridge motor driver (L298N) is used with an **Arduino Uno** to control motor direction, while PWM (Pulse Width Modulation) is used to control motor speed. Push buttons are implemented for manual control, and LEDs provide visual feedback of motor operation.

3. Literature Review

DC Motor Speed Control

Motor speed is proportional to the average voltage applied. PWM is used to vary this voltage efficiently by adjusting duty cycle.

Switching Devices (Driver IC)

The L298N motor driver acts as a switching device that allows low-power signals from Arduino to control high-current motors.

H-Bridge Concept

An H-Bridge allows current to flow in both directions through the motor, enabling forward and reverse rotation.

Flyback Protection

Although internal diodes are present in L298N, generally **flyback diodes** are used to protect circuits from voltage spikes caused by inductive loads.

4. Components Required

- Arduino Uno R3
- L298N Motor Driver Module
- DC Motor
- 9V Battery
- Push Buttons (2)
- LEDs (2)
- Resistors (10k Ω , 220 Ω)
- Breadboard
- Jumper wires
- Slide switch

5. Circuit Description

5.1 Power Distribution

- Common ground is maintained between Arduino, battery, and L298N.
- Arduino provides 5V logic supply.
- External 9V battery powers the motor through the driver.

5.2 Motor Driver Interface

- ENA → Pin 10 (PWM for speed control)
- IN1 → Pin 8
- IN2 → Pin 9
- Motor connected to OUT1 and OUT2

5.3 Input System

- Button 1 (Forward) → Pin 2
- Button 2 (Reverse) → Pin 3
- Both use 10k Ω pull-down resistors

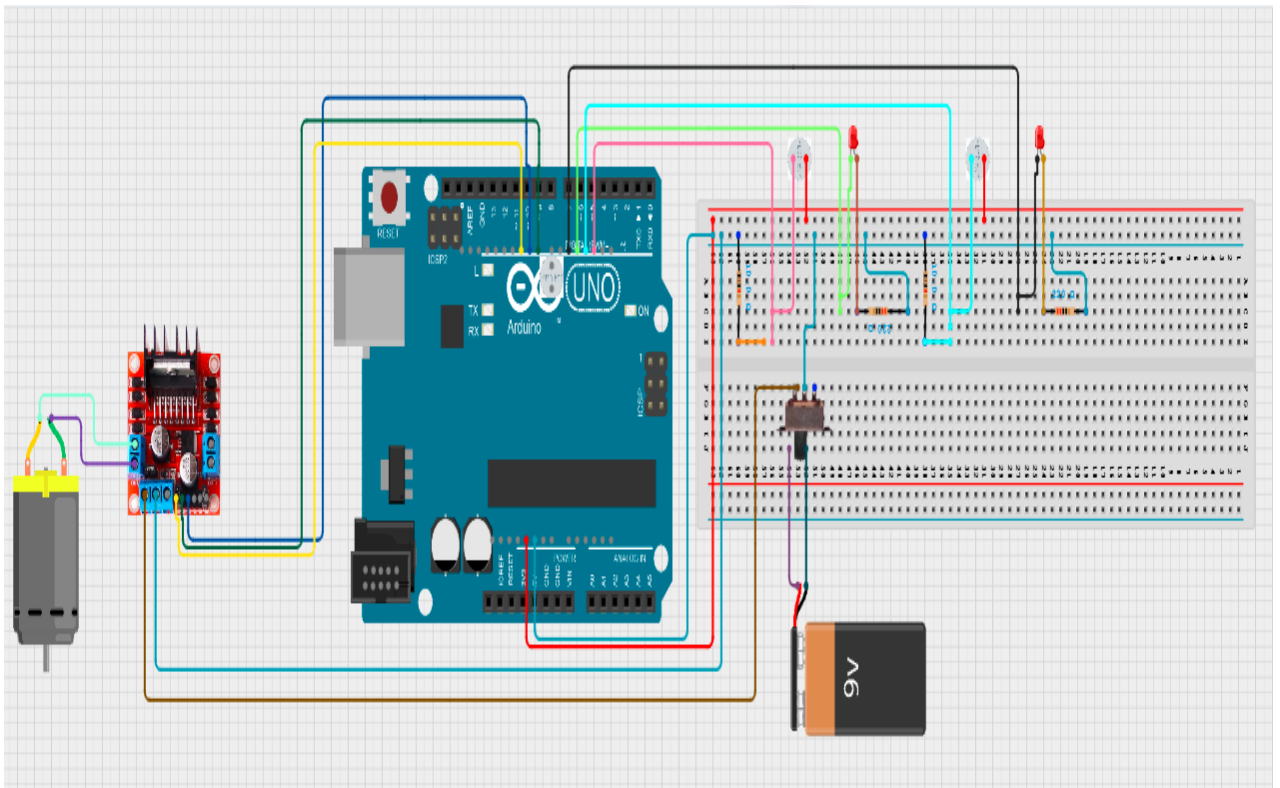
5.4 Output Indicators

- Forward LED → Pin 4
- Reverse LED → Pin 5
- Each uses 220 Ω resistor

6. Circuit Operation

- When Forward button is pressed:
 - IN1 = HIGH, IN2 = LOW
 - Motor rotates forward
 - Forward LED turns ON
- When Reverse button is pressed:
 - IN1 = LOW, IN2 = HIGH
 - Motor rotates in reverse
 - Reverse LED turns ON
- Speed is controlled by PWM signal applied at ENA pin.

Schematics



7. Arduino Code

sketch_apr10a.ino

```
1  const int buttonForward = 2;
2  const int buttonReverse = 3;
3  const int ENA = 10;
4  const int IN1 = 8;
5  const int IN2 = 9;
6  const int ledForward = 4;
7  const int ledReverse = 5;
8
9  int motorSpeed = 200;
10 int motorStatus = 0; // 0=Stop, 1=Forward, 2=Reverse
11
12 void setup() {
13   pinMode(buttonForward, INPUT);
14   pinMode(buttonReverse, INPUT);
15   pinMode(ENA, OUTPUT);
16   pinMode(IN1, OUTPUT);
17   pinMode(IN2, OUTPUT);
18   pinMode(ledForward, OUTPUT);
19   pinMode(ledReverse, OUTPUT);
20   stopMotor();
21 }
22
23 void loop() {
24   // --- Forward Button Logic ---
25   if (digitalRead(buttonForward) == HIGH) {
26     delay(50); // Small debounce
27     if (motorStatus == 1) {
28       stopMotor();
29     } else {
30       moveForward();
31     }
32     while(digitalRead(buttonForward) == HIGH);
33     delay(50);
34   }
35
36   // --- Reverse Button Logic ---
37   if (digitalRead(buttonReverse) == HIGH) {
38     delay(50);
39     if (motorStatus == 2) {
40       stopMotor();
41     } else {
42       moveReverse();
43     }
44     while(digitalRead(buttonReverse) == HIGH);
45     delay(50);
46   }
47 }
```

```

48
49 void moveForward() {
50     digitalWrite(IN1, HIGH);
51     digitalWrite(IN2, LOW);
52     analogWrite(ENA, motorSpeed);
53     digitalWrite(ledForward, HIGH);
54     digitalWrite(ledReverse, LOW);
55     motorStatus = 1;
56 }
57
58 void moveReverse() {
59     digitalWrite(IN1, LOW);
60     digitalWrite(IN2, HIGH);
61     analogWrite(ENA, motorSpeed);
62     digitalWrite(ledForward, LOW);
63     digitalWrite(ledReverse, HIGH);
64     motorStatus = 2;
65 }
66
67 void stopMotor() {
68     digitalWrite(IN1, LOW);
69     digitalWrite(IN2, LOW);
70     analogWrite(ENA, 0);
71     digitalWrite(ledForward, LOW);
72     digitalWrite(ledReverse, LOW);
73     motorStatus = 0;
74 }

```

8. Circuit Working

1. System Initialization

At the start of the system, power is supplied to both the control and driving sections. The Arduino initializes all input and output pins, setting up communication with the push buttons, LEDs, and the motor driver module. Meanwhile, the L298N motor driver receives its motor supply from the external battery, ensuring sufficient current is available for motor operation.

2. Idle State (No Input Condition)

Initially, the system remains in an idle state. Both control inputs are LOW due to the pull-down resistors, so the motor remains stationary and both LEDs stay OFF, indicating no active command.

3. Forward Operation

When the forward push button is pressed, a HIGH signal is sent to the Arduino input pin. The Arduino processes this input and responds by sending appropriate logic signals to the motor driver: one input pin is set HIGH while the other is set LOW. This creates a current path through the H-Bridge in one direction.

As a result, the motor begins rotating in the forward direction. Simultaneously, the Arduino activates the forward indicator LED, providing visual confirmation of the motor's state.

4. Speed Control using PWM

At the same time, the Arduino outputs a PWM signal to the enable pin of the motor driver. This PWM signal rapidly switches the motor ON and OFF at a high frequency. By controlling the duty cycle of this signal, the average voltage applied to the motor is adjusted, which directly controls its speed.

A higher duty cycle results in a faster motor speed, while a lower duty cycle reduces the speed.

5. Reverse Operation

When the reverse push button is pressed instead, the Arduino detects this new input and reverses the logic signals sent to the motor driver. The polarity across the motor terminals is flipped, causing the current to flow in the opposite direction.

Consequently, the motor begins rotating in reverse. The reverse LED is turned ON to indicate this change in direction.

6. Motor Stop Condition

If neither button is pressed, the Arduino ensures that both control inputs to the motor driver are LOW. This stops current flow through the motor, bringing it to a halt. The LEDs are also turned OFF, indicating that the system is inactive.

7. System Stability and Protection

Throughout operation, the common grounding ensures stable reference levels between all components, minimizing noise and ensuring reliable signal transmission. The motor driver safely handles the higher current demands of the motor, protecting the Arduino from potential damage.

8. Overall Working Summary

In this way, the system smoothly transitions between idle, forward, and reverse states while allowing real-time speed control through PWM.

9. Relevant Equations

$$\text{RPM} = \frac{V_{\text{measured}}}{6V} \times 13,000$$

$$\text{Duty Cycle} = \frac{\text{PWM Value}}{255} \times 100$$

10. Observations / Results

Duty Cycle (%)	PWM Frequency (Hz)	Motor Speed (RPM)	Direction	Voltage (V)	Remarks
78.4%	~490 Hz	~8,016	Forward/Reverse	± 3.7V	Battery voltage sag observed
58.8%	~490 Hz	~6,283	Forward/Reverse	± 2.9V	Speed proportional to voltage
47.1%	~490 Hz	~5,417	Forward/Reverse	± 2.5V	Near minimum operating torque

Note: The negative voltage shows that the motor direction is reversed.

10. Analysis

- Increasing PWM duty cycle increases motor speed.
- Direction control is successfully achieved using H-Bridge logic.
- LEDs accurately indicate motor state.
- System performs reliably with stable grounding.

11. Experiential Learning

- Learned practical implementation of H-Bridge motor control.
- Understood how PWM controls speed efficiently.
- Gained experience in Arduino programming and interfacing.
- Observed real-world issues like noise and grounding importance.

12. Conclusion

The project successfully demonstrates control of a DC motor's speed and direction using Arduino and an L298N driver. PWM proved effective for speed control, while digital logic controlled the direction efficiently. The system is reliable and suitable for real-world automation applications.